Pandas Full Course Notes — Beginner to Advanced

Python | Bangla Explanation | Practical Examples

---

1. Introduction to Pandas

What is Pandas?

Pandas হলো Python-এর একটি powerful data analysis এবং data manipulation library।

এটি ব্যবহার করে:

- Data cleaning
- Data analysis
- Data filtering
- Data transformation
- Missing value handling
- CSV, Excel, JSON files processing
- Database data analysis
- Time series analysis

Install Pandas

```
pip install pandas
```

Import Pandas

```
import pandas as pd
import numpy as np
```

`pd` হলো Pandas-এর standard alias।

---

2. Pandas Main Data Structures

Pandas-এর প্রধান দুটি data structure:

1. Series
2. DataFrame

2.1 Series

Series হলো এক-dimensional labeled array।

```
import pandas as pd

data = [10, 20, 30, 40]

series = pd.Series(data)

print(series)
```

Output:

```
0    10
1    20
2    30
3    40
dtype: int64
```

Custom Index

```
series = pd.Series(
    [80, 90, 75],
    index=["Rahim", "Karim", "Faruk"]
)

print(series)
```

Access Series Elements

```
print(series["Rahim"])
print(series.iloc[0])
```

Series Attributes

```
print(series.index)
print(series.values)
print(series.dtype)
print(series.shape)
print(series.size)
print(series.ndim)
```

---

3. DataFrame

DataFrame হলো two-dimensional labeled tabular data structure।

এটি Excel spreadsheet বা SQL table-এর মতো।

Create DataFrame

```
import pandas as pd

data = {
    "name": ["Faruk", "Rahim", "Karim"],
    "age": [25, 26, 24],
    "salary": [30000, 35000, 28000]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
    name  age  salary
0  Faruk   25   30000
1  Rahim   26   35000
2  Karim   24   28000
```

Custom Index

```
df = pd.DataFrame(
    data,
    index=["A", "B", "C"]
)
```

```
print(df)
```

DataFrame Attributes

```
print(df.shape)
print(df.size)
print(df.ndim)
print(df.columns)
print(df.index)
print(df.dtypes)
print(df.values)
```

Attribute	Meaning
shape	Rows and columns
size	Total number of cells
ndim	Number of dimensions
columns	Column names
index	Row labels
dtypes	Data types
values	Underlying values

---

4. Reading Data from Files

Pandas বিভিন্ন file format read করতে পারে।

Read CSV

```
df = pd.read_csv("employees.csv")

print(df)
```

Read Excel

```
df = pd.read_excel("employees.xlsx")
```

Excel support-এর জন্য প্রয়োজন হলে:

```
pip install openpyxl
```

Read JSON

```
df = pd.read_json("employees.json")
```

Read SQL Database

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine("sqlite:///company.db")

df = pd.read_sql("SELECT * FROM employees", engine)

print(df)
```

---

5. Inspecting Data

Data analysis শুরু করার আগে dataset inspect করা জরুরি।

head()

প্রথম পাঁচটি row দেখায়।

```
df.head()
```

Custom rows:

```
df.head(10)
```

tail()

শেষ পাঁচটি row দেখায়।

```
df.tail()
```

sample()

Random rows দেখায়।

```
df.sample(5)
```

info()

Column, data type এবং non-null count দেখায়।

```
df.info()
```

describe()

Numerical columns-এর statistical summary দেখায়।

```
df.describe()
```

সব column:

```
df.describe(include="all")
```

Other Inspection Methods

```
print(df.shape)
print(df.columns)
print(df.index)
print(df.dtypes)
print(df.nunique())
print(df.memory_usage(deep=True))
```

---

6. Selecting Columns

Single Column

```
names = df["name"]

print(names)
```

এটি Series return করে।

Multiple Columns

```
result = df[["name", "salary"]]

print(result)
```

এটি DataFrame return করে।

Column Using Dot Notation

```
print(df.name)
```

Dot notation সব column name-এর জন্য কাজ করে না। সাধারণত bracket notation বেশি reliable।

---

7. Selecting Rows — loc and iloc

এ দুটি খুব গুরুত্বপূর্ণ।

loc[]

Label-based selection।

```
print(df.loc[0])
```

Multiple rows:

```
print(df.loc[0:2])
```

Specific rows and columns:

```
print(df.loc[0:2, ["name", "salary"]])
```

iloc[]

Position-based selection।

```
print(df.iloc[0])
```

First two rows:

```
print(df.iloc[0:2])
```

Specific row and column positions:

```
print(df.iloc[0:2, 0:2])
```

loc vs iloc

loc	iloc
Label-based	Position-based
Label ব্যবহার করে	Integer position ব্যবহার করে
Slice-এর end সাধারণত included	Slice-এর end excluded

---

8. Filtering Data

Condition অনুযায়ী rows select করা যায়।

```
df = pd.DataFrame({
    "name": ["Faruk", "Rahim", "Karim", "Sakib"],
    "age": [25, 30, 22, 35],
    "salary": [30000, 50000, 25000, 60000]
})
```

Greater Than

```
result = df[df["salary"] > 30000]

print(result)
```

Less Than

```
result = df[df["age"] < 30]
```

Multiple Conditions

AND:

```
result = df[
    (df["age"] > 25) &
    (df["salary"] > 30000)
]
```

OR:

```
result = df[
    (df["age"] < 25) |
```

```
    (df["salary"] > 50000)
]
```

NOT:

```
result = df[~(df["age"] > 30)]
```

isin()

```
result = df[df["name"].isin(["Faruk", "Rahim"])]
```

between()

```
result = df[df["age"].between(20, 30)]
```

query()

```
result = df.query("age > 25 and salary > 30000")
```

---

9. Adding New Columns

Direct Assignment

```
df["department"] = ["IT", "HR", "Sales", "IT"]
```

Calculated Column

```
df["bonus"] = df["salary"] * 0.10
```

Multiple Conditions with np.where()

```
import numpy as np

df["status"] = np.where(
    df["salary"] >= 40000,
    "High Salary",
    "Normal Salary"
)
```

Using assign()

```
new_df = df.assign(
    annual_salary=df["salary"] * 12
)
```

`assign()` নতুন DataFrame return করে।

---

10. Updating Data

Update a Value

```
df.loc[0, "salary"] = 40000
```

Update Multiple Rows

```
df.loc[df["salary"] < 30000, "salary"] = 30000
```

Rename Columns

```
df = df.rename(columns={
    "name": "employee_name",
    "salary": "monthly_salary"
})
```

Rename Index

```
df = df.rename(index={
    0: "Employee_1",
    1: "Employee_2"
})
```

---

11. Deleting Rows and Columns

Drop Column

```
df = df.drop(columns=["bonus"])
```

Drop Multiple Columns

```
df = df.drop(columns=["bonus", "status"])
```

Drop Row

```
df = df.drop(index=0)
```

Reset Index

```
df = df.reset_index(drop=True)
```

`drop()` default-ভাবে original DataFrame পরিবর্তন করে না; নতুন DataFrame return করে।

---

12. Sorting Data

Sort by One Column

```
df = df.sort_values("salary")
```

Descending:

```
df = df.sort_values("salary", ascending=False)
```

Sort by Multiple Columns

```
df = df.sort_values(
    ["department", "salary"],
    ascending=[True, False]
)
```

Sort by Index

```
df = df.sort_index()
```

---

13. Handling Missing Values

Real-world dataset-এ missing values খুব common।

Create Dataset with Missing Values

```
df = pd.DataFrame({
    "name": ["Faruk", "Rahim", None, "Karim"],
    "age": [25, np.nan, 30, 22],
    "salary": [30000, 40000, np.nan, 25000]
})
```

Detect Missing Values

```
print(df.isna())
print(df.isnull())
```

Count Missing Values

```
print(df.isna().sum())
```

Total Missing Values

```
print(df.isna().sum().sum())
```

Remove Missing Rows

```
df = df.dropna()
```

Remove Columns with Missing Values

```
df = df.dropna(axis=1)
```

Fill Missing Values

```
df["age"] = df["age"].fillna(0)
```

Fill with mean:

```
df["age"] = df["age"].fillna(
    df["age"].mean()
)
```

Fill with median:

```
df["salary"] = df["salary"].fillna(
    df["salary"].median()
)
```

Fill with mode:

```
df["name"] = df["name"].fillna(
    df["name"].mode()[0]
)
```

Forward Fill

```
df = df.ffill()
```

Backward Fill

```
df = df.bfill()
```

Important: Missing value fill করার আগে dataset-এর business meaning বুঝতে হবে। সব missing values zero দিয়ে fill করা ঠিক নয়।

---

14. Handling Duplicate Values

Check Duplicates

```
print(df.duplicated())
```

Count Duplicates

```
print(df.duplicated().sum())
```

Remove Duplicates

```
df = df.drop_duplicates()
```

Based on Specific Column

```
df = df.drop_duplicates(
    subset=["name"]
)
```

Keep last duplicate:

```
df = df.drop_duplicates(
    subset=["name"],
    keep="last"
)
```

---

15. Data Type Conversion

Check Data Types

```
print(df.dtypes)
```

Convert to Integer

```
df["age"] = df["age"].astype("int64")
```

Missing values থাকলে nullable integer dtype ব্যবহার করা যায়:

```
df["age"] = df["age"].astype("Int64")
```

Convert to Float

```
df["salary"] = df["salary"].astype(float)
```

Convert to String

```
df["name"] = df["name"].astype("string")
```

Convert to DateTime

```
df["date"] = pd.to_datetime(df["date"])
```

Safe Numeric Conversion

```
df["salary"] = pd.to_numeric(
    df["salary"],
    errors="coerce"
)
```

Invalid values NaN হয়ে যাবে।

---

16. String Operations

Pandas string methods ব্যবহার করে text cleaning করা যায়।

```
df = pd.DataFrame({
    "name": [" faruk ", "RAHIM", "karim"]
})
```

Lowercase

```
df["name"] = df["name"].str.lower()
```

Uppercase

```
df["name"] = df["name"].str.upper()
```

Title Case

```
df["name"] = df["name"].str.title()
```

Remove Spaces

```
df["name"] = df["name"].str.strip()
```

Replace Text

```
df["name"] = df["name"].str.replace(
    "faruk",
    "omar",
    regex=False
)
```

Contains

```
result = df[
    df["name"].str.contains(
        "rahim",
        case=False,
        na=False
    )
]
```

Starts With

```
result = df[
    df["name"].str.startswith("f", na=False)
]
```

Split

```
df = pd.DataFrame({
    "full_name": ["Faruk Ahmed", "Rahim Uddin"]
})

df[["first_name", "last_name"]] = (
    df["full_name"].str.split(" ", n=1, expand=True)
)
```

---

17. Apply, Map and Applymap

apply()

Series-এর প্রতিটি value বা DataFrame-এর axis অনুযায়ী function apply করে।

```
df = pd.DataFrame({
    "salary": [20000, 30000, 40000]
})

df["bonus"] = df["salary"].apply(
    lambda x: x * 0.10
)
```

map()

Series-এর values map করে।

```
df = pd.DataFrame({
    "status": ["active", "inactive", "active"]
})

df["status_code"] = df["status"].map({
    "active": 1,
    "inactive": 0
})
```

DataFrame.map()

Element-wise mapping:

```
df = pd.DataFrame({
    "a": [1, 2],
    "b": [3, 4]
})

result = df.map(lambda x: x * 2)
```

Note: পুরোনো Pandas version-এ element-wise DataFrame mapping-এর জন্য `applymap()` দেখা যেতে পারে। Current versions-এ `DataFrame.map()` ব্যবহার করো।

Performance tip: সাধারণ arithmetic বা string operation-এর ক্ষেত্রে built-in vectorized methods সাধারণত `apply()`-এর চেয়ে ভালো।

---

18. GroupBy Operations

GroupBy হলো Pandas-এর সবচেয়ে গুরুত্বপূর্ণ concepts-এর একটি।
এটি split → apply → combine pattern অনুসরণ করে।

```
df = pd.DataFrame({
    "department": ["IT", "HR", "IT", "HR", "Sales"],
    "salary": [30000, 25000, 40000, 35000, 50000]
})
```

Group by Department

```
result = df.groupby("department")["salary"].mean()

print(result)
```

Sum

```
print(
    df.groupby("department")["salary"].sum()
)
```

Count

```
print(
    df.groupby("department")["salary"].count()
)
```

Multiple Aggregations

```
result = df.groupby("department").agg({
    "salary": ["sum", "mean", "max", "min", "count"]
})

print(result)
```

Named Aggregation

```
result = df.groupby("department").agg(
    total_salary=("salary", "sum"),
    average_salary=("salary", "mean"),
    employee_count=("salary", "count")
)
```

Multiple Grouping Columns

```
result = df.groupby(
    ["department", "city"]
)["salary"].mean()
```

Transform

Original row structure বজায় রেখে group-wise calculation করে।

```
df["department_avg"] = (
    df.groupby("department")["salary"]
```

```
      .transform("mean")
)
```

Filter

Group-এর condition অনুযায়ী groups select করে।

```
result = df.groupby("department").filter(
    lambda group: group["salary"].mean() > 30000
)
```

---

19. Merging DataFrames

SQL JOIN-এর মতো কাজ করে।

```
employees = pd.DataFrame({
    "employee_id": [1, 2, 3],
    "name": ["Faruk", "Rahim", "Karim"]
})

departments = pd.DataFrame({
    "employee_id": [1, 2, 4],
    "department": ["IT", "HR", "Sales"]
})
```

Inner Join

```
result = pd.merge(
    employees,
    departments,
    on="employee_id",
    how="inner"
)
```

Left Join

```
result = pd.merge(
    employees,
    departments,
    on="employee_id",
    how="left"
)
```

Right Join

```
result = pd.merge(
    employees,
    departments,
    on="employee_id",
    how="right"
)
```

Outer Join

```
result = pd.merge(
    employees,
```

```
    departments,
    on="employee_id",
    how="outer"
)
```

Join Types

Join	Meaning
inner	Matching rows only
left	All left rows
right	All right rows
outer	All rows from both sides

Different Column Names

```
pd.merge(
    employees,
    departments,
    left_on="employee_id",
    right_on="emp_id"
)
```

Validate Merge Relationships

```
pd.merge(
    employees,
    departments,
    on="employee_id",
    how="left",
    validate="one_to_one"
)
```

`validate` unexpected duplicate-key relationships detect করতে সাহায্য করে।

---

20. Concatenation

Multiple DataFrames combine করার জন্য `pd.concat()`।

Vertical Concatenation

```
df1 = pd.DataFrame({
    "name": ["Faruk", "Rahim"]
})
```

```
df2 = pd.DataFrame({
    "name": ["Karim", "Sakib"]
})

result = pd.concat(
    [df1, df2],
    ignore_index=True
)
```

Horizontal Concatenation

```
result = pd.concat(
    [df1, df2],
    axis=1
)
```

Join Different Columns

```
result = pd.concat(
    [df1, df2],
    ignore_index=True,
    sort=False
)
```

---

21. Pivot Tables

Pivot table দিয়ে grouped summary তৈরি করা যায়।

```
df = pd.DataFrame({
    "department": ["IT", "IT", "HR", "HR"],
    "gender": ["Male", "Female", "Male", "Female"],
    "salary": [30000, 35000, 25000, 30000]
})
```

```
result = pd.pivot_table(
    df,
    values="salary",
    index="department",
    columns="gender",
    aggfunc="mean"
)

print(result)
```

Multiple Aggregations

```
result = pd.pivot_table(
    df,
    values="salary",
    index="department",
    aggfunc=["mean", "sum", "max"]
)
```

pivot() vs pivot_table()

- `pivot()` → Duplicate index-column combinations থাকলে error হয়।

- `pivot_table()` → Aggregation করে duplicate combinations handle করতে পারে।

---

22. Crosstab

Frequency table তৈরি করে।

```
df = pd.DataFrame({
    "gender": ["Male", "Female", "Male", "Female"],
    "department": ["IT", "HR", "HR", "IT"]
})

result = pd.crosstab(
    df["gender"],
    df["department"]
)

print(result)
```

Normalize:

```
result = pd.crosstab(
    df["gender"],
    df["department"],
    normalize="index"
)
```

---

23. Working with Dates and Time Series

Convert to DateTime

```
df = pd.DataFrame({
    "date": ["2026-01-01", "2026-01-02", "2026-01-03"],
    "sales": [100, 200, 300]
})

df["date"] = pd.to_datetime(df["date"])
```

Extract Year

```
df["year"] = df["date"].dt.year
```

Extract Month

```
df["month"] = df["date"].dt.month
```

Extract Day

```
df["day"] = df["date"].dt.day
```

Day Name

```
df["day_name"] = df["date"].dt.day_name()
```

Date Filtering

```
result = df[
    df["date"] >= "2026-01-02"
]
```

Set Date as Index

```
df = df.set_index("date")
```

Resampling

```
daily_sales = df["sales"].resample("D").sum()
```

Monthly:

```
monthly_sales = df["sales"].resample("ME").sum()
```

`ME` হলো month-end frequency alias।

---

24. Rolling and Expanding Windows

Rolling Mean

শেষ তিনটি observation-এর average।

```
df = pd.DataFrame({
    "sales": [100, 200, 300, 400, 500]
})

df["rolling_avg"] = (
    df["sales"].rolling(window=3).mean()
)
```

Rolling Sum

```
df["rolling_sum"] = (
    df["sales"].rolling(window=3).sum()
)
```

Expanding Mean

শুরু থেকে বর্তমান observation পর্যন্ত average।

```
df["expanding_avg"] = (
    df["sales"].expanding().mean()
)
```

---

25. Window Functions

shift()

Previous row-এর value নিতে।

```
df["previous_sales"] = df["sales"].shift(1)
```

diff()

Current এবং previous row-এর difference।

```
df["sales_change"] = df["sales"].diff()
```

pct_change()

Percentage change:

```
df["sales_growth"] = df["sales"].pct_change()
```

rank()

```
df["rank"] = df["sales"].rank(
    ascending=False
)
```

---

26. Categorical Data

Repeated text values-এর memory usage কমাতে category dtype ব্যবহার করা যায়।

```
df = pd.DataFrame({
    "department": ["IT", "HR", "IT", "Sales"]
})

df["department"] = df["department"].astype("category")
```

Check:

```
print(df["department"].dtype)
```

Categorical values:

```
print(df["department"].cat.categories)
```

---

27. Reshaping Data

melt()

Wide format থেকে long format।

```
df = pd.DataFrame({
    "name": ["Faruk", "Rahim"],
    "January": [100, 200],
    "February": [150, 250]
})

result = pd.melt(
    df,
    id_vars=["name"],
    var_name="month",
    value_name="sales"
)

print(result)
```

stack()

Columns-কে index-এর দিকে নিয়ে যায়।

```
result = df.set_index("name").stack()
```

unstack()

Index level-কে column-এ নিয়ে যায়।

```
result = result.unstack()
```

---

28. Exporting Data

Save CSV

```
df.to_csv("output.csv", index=False)
```

Save Excel

```
df.to_excel("output.xlsx", index=False)
```

Save JSON

```
df.to_json("output.json", orient="records", indent=2)
```

Save HTML

```
df.to_html("output.html", index=False)
```

---

29. SQL and Pandas

SQL to DataFrame

```
from sqlalchemy import create_engine

engine = create_engine("sqlite:///company.db")

df = pd.read_sql(
    "SELECT * FROM employees",
    engine
)
```

DataFrame to SQL

```
df.to_sql(
    "employees",
    engine,
    if_exists="replace",
    index=False
)
```

Possible `if_exists` values:

- `fail`
- `replace`
- `append`

---

30. Advanced Performance Optimization

1. Avoid Unnecessary Loops

Less efficient:

```
result = []

for salary in df["salary"]:
    result.append(salary * 2)
```

Better:

```
df["double_salary"] = df["salary"] * 2
```

2. Use Appropriate Data Types

```
df["age"] = df["age"].astype("int32")
```

3. Use Category for Repeated Labels

```
df["department"] = df["department"].astype("category")
```

4. Read Large CSV in Chunks

```
for chunk in pd.read_csv(
    "large_file.csv",
    chunksize=10000
):
    result = chunk["salary"].sum()
    print(result)
```

5. Select Only Required Columns

```
df = pd.read_csv(
    "employees.csv",
    usecols=["name", "salary"]
)
```

6. Use Efficient Filtering

```
result = df.loc[df["salary"] > 30000]
```

7. Avoid Repeated Concatenation in Loops

Better:

```
frames = []

for file in files:
    frames.append(pd.read_csv(file))

result = pd.concat(frames, ignore_index=True)
```

---

31. Advanced Missing Data Techniques

Interpolation

```
df = pd.DataFrame({
    "sales": [100, np.nan, 300, np.nan, 500]
})
```

```
df["sales"] = df["sales"].interpolate()
```

Drop Rows Based on Specific Columns

```
df = df.dropna(
    subset=["name", "salary"]
)
```

Fill Different Columns Differently

```
df = df.fillna({
    "age": df["age"].median(),
    "salary": 0,
    "department": "Unknown"
})
```

---

32. Advanced GroupBy Techniques

Multiple Aggregations

```
result = df.groupby("department").agg(
    total_salary=("salary", "sum"),
    average_salary=("salary", "mean"),
    maximum_salary=("salary", "max")
)
```

Group-wise Ranking

```
df["department_rank"] = (
    df.groupby("department")["salary"]
      .rank(ascending=False)
)
```

Group-wise Cumulative Sum

```
df["cumulative_salary"] = (
    df.groupby("department")["salary"]
      .cumsum()
)
```

Group-wise Difference

```
df["salary_difference"] = (
    df.groupby("department")["salary"]
      .diff()
)
```

---

33. Data Cleaning Project

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
```

```
    "name": [" Faruk ", "Rahim", "Karim", "Faruk "],
    "age": [25, np.nan, 30, 25],
    "salary": [30000, 40000, np.nan, 30000],
    "department": ["IT", "HR", "IT", "IT"]
})

# Remove extra spaces
df["name"] = df["name"].str.strip()

# Fill missing age
df["age"] = df["age"].fillna(df["age"].median())

# Fill missing salary
df["salary"] = df["salary"].fillna(df["salary"].median())

# Remove duplicates
df = df.drop_duplicates()

# Reset index
df = df.reset_index(drop=True)

# Add annual salary
df["annual_salary"] = df["salary"] * 12

print(df)
```

---

34. Real-World Project: Sales Analysis

```
import pandas as pd

sales = pd.DataFrame({
    "date": [
        "2026-01-01",
        "2026-01-02",
        "2026-01-03",
        "2026-01-04",
        "2026-01-05"
    ],
    "product": [
        "Laptop",
        "Mobile",
        "Laptop",
        "Keyboard",
        "Mobile"
    ],
    "quantity": [2, 3, 1, 5, 2],
    "price": [50000, 20000, 50000, 1500, 20000]
})

# Convert date
sales["date"] = pd.to_datetime(sales["date"])

# Calculate revenue
```

```
sales["revenue"] = sales["quantity"] * sales["price"]

# Total revenue
print("Total Revenue:", sales["revenue"].sum())

# Product-wise revenue
print(
    sales.groupby("product")["revenue"].sum()
)

# Daily revenue
print(
    sales.groupby("date")["revenue"].sum()
)

# Highest revenue product
print(
    sales.groupby("product")["revenue"]
         .sum()
         .idxmax()
)
```

---

35. Pandas with NumPy

Pandas এবং NumPy একসঙ্গে কাজ করে।

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "marks": [80, 70, 90, 60]
})

df["grade"] = np.where(
    df["marks"] >= 80,
    "A",
    "B"
)

print(df)
```

DataFrame থেকে NumPy array:

```
arr = df["marks"].to_numpy()
```

NumPy array থেকে DataFrame:

```
new_df = pd.DataFrame(arr, columns=["marks"])
```

---

36. Pandas Interview Questions

Beginner

1. What is Pandas?

Python-এর data analysis and manipulation library।

2. What is Series?

One-dimensional labeled data structure।

3. What is DataFrame?

Two-dimensional tabular data structure।

4. What is the difference between Series and DataFrame?

Series এক-dimensional; DataFrame two-dimensional।

5. How do you read a CSV file?

```
pd.read_csv("data.csv")
```

6. How do you inspect a DataFrame?

```
df.head()
df.info()
df.describe()
```

7. What is loc vs iloc?

`loc` label-based এবং `iloc` position-based।

Intermediate

8. How do you handle missing values?

`isna()`, `dropna()`, `fillna()`।

9. What is groupby?

Group-wise aggregation ও transformation করার পদ্ধতি।

10. What is merge?

দুটি DataFrame-এর matching keys দিয়ে combine করা।

11. What is concat?

একাধিক DataFrame একসঙ্গে যুক্ত করা।

12. What is pivot_table?

Data summarize করার জন্য grouped table তৈরি করে।

13. What is apply?

Series বা DataFrame-এর ওপর function apply করে।

14. What is map?

Series-এর values mapping বা transformation করে।

Advanced

15. What is transform?

Group-wise calculation করে original row structure বজায় রাখে।

16. What is rolling?

Moving window calculation করে।

17. What is resample?

Time-series data-কে নতুন frequency-তে aggregate করে।

18. What is categorical dtype?

Repeated category values efficiently store করার dtype।

19. How do you process large CSV files?

`chunksize` ব্যবহার করে chunk-by-chunk processing করা যায়।

20. How do you optimize Pandas performance?

- Vectorized operations
- Efficient dtypes
- Category dtype
- Column selection
- Chunk processing
- Avoid unnecessary copies and loops

---

37. Practice Problems

Beginner Level

1. Create a DataFrame with name, age and salary.
2. Display first five rows.
3. Display last three rows.
4. Find the number of rows and columns.
5. Select one column.
6. Select multiple columns.
7. Select rows using `loc`.
8. Select rows using `iloc`.
9. Filter employees with salary above 30000.
10. Sort employees by age.

Intermediate Level

1. Find missing values in every column.
2. Fill missing salary with median.
3. Remove duplicate rows.
4. Rename DataFrame columns.
5. Add a bonus column.
6. Group employees by department.
7. Calculate average salary by department.
8. Merge employee and department tables.
9. Create a pivot table.
10. Convert a date column to datetime.

Advanced Level

1. Calculate monthly sales using resampling.
2. Calculate a 7-day rolling average.
3. Find the top three employees in each department.
4. Calculate department-wise cumulative salary.
5. Convert wide data into long data using melt.
6. Process a large CSV in chunks.
7. Optimize a DataFrame's memory usage.
8. Find duplicate keys before merging.
9. Create a complete data-cleaning pipeline.
10. Perform end-to-end sales analysis.

---

38. Complete Pandas Learning Roadmap

Level 1 — Beginner

- Pandas introduction
- Series
- DataFrame
- DataFrame creation
- CSV and Excel reading
- Data inspection
- Selecting columns
- loc and iloc
- Filtering
- Sorting
- Adding and removing columns

Level 2 — Intermediate

- Missing values
- Duplicate handling
- Data type conversion
- String operations
- Apply and map
- GroupBy
- Merge
- Join
- Concat
- Pivot tables
- Crosstab

Level 3 — Advanced

- DateTime
- Time series
- Rolling and expanding
- Shift and diff
- Categorical data
- Reshaping
- Melt, stack and unstack
- Advanced GroupBy
- SQL integration
- Performance optimization

Level 4 — Expert

- Large dataset processing
- Chunk processing
- Memory optimization
- Advanced indexing
- Efficient data pipelines
- Complex data transformations

- Time-series analytics
- Data quality validation
- Pandas with NumPy and SciPy

---

39. Recommended Study Routine

Daily 2 hours:

Time	Activity
30 min	Learn concepts
30 min	Practice code
30 min	Solve problems
30 min	Build mini project

Recommended projects:

1. Student Result Analysis
2. Employee Salary Analysis
3. E-commerce Sales Analysis
4. Customer Data Cleaning
5. Monthly Revenue Dashboard
6. Real-world CSV Data Analysis

Final Advice

Pandas শেখার সময় বিশেষভাবে ভালোভাবে আয়ত্ত করো:

1. Series and DataFrame
2. loc and iloc
3. Boolean filtering
4. Missing value handling
5. GroupBy
6. Merge and Join
7. Pivot tables
8. DateTime operations
9. Rolling windows
10. Performance optimization

Final Goal: Raw CSV/Excel/database data থেকে clean, transform, analyze এবং useful insights বের করতে পারা।